University Project-
Step 1:

```python
import csv
import pandas as pd

# Step a: Define station names
stations = ["ML0025", "ML0029", "ML0037"]

# Step b: Initialize row counter
counter = 0

# Step c: Empty the output file
with open("project_data.csv", "w"):
    pass

# Step d: Loop over years and quarters
for i in range(2014, 2020):
    for j in range(1, 5):
        filename = f"{i}-Q{j}-Central.csv"
        try:
            with open(filename, "r") as csvfile:
                reader = csv.reader(csvfile)
                for row in reader:
                    if row[1] in stations:
                        row[0] = f"{i}Q{j}"  # Step g.1: Update first entry
                        out_row = [counter] + row  # Step g.2: Add counter to row
                        counter += 1

                        # Step g.3: Append to project_data.csv
                        with open("project_data.csv", "a", newline="") as outfile:
                            writer = csv.writer(outfile)
                            writer.writerow(out_row)
        except FileNotFoundError:
            print(f"Warning: {filename} not found, skipping.")

# Step h: Define column names
col_names = ["Quarter", "Station", "Date", "Weather", "Time", "Day", "Drop1", "Direction", "Drop2", "Mode", "Count"]

# Step i: Read CSV into pandas dataframe
df = pd.read_csv("project_data.csv", names=col_names)

# Step j: Drop unnecessary columns
df = df.drop(columns=["Drop1", "Drop2"], errors='ignore')

# Step k: Create Full_time column
df["Full_time"] = df["Date"] + " " + df["Time"]

# Step l: Output to Excel
df.to_excel("CycleData.xlsx", sheet_name="Sheet1", index=False)
```

```python
# Step 1: Output to Excel
df.to_excel("CycleData.xlsx", sheet_name="Sheet1", index=False)

print("Processing complete. Data as CycleData.xlsx.")
```

Step 2:

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the data
file_path = "CycleData.xlsx"
my_data = pd.read_excel(file_path, index_col=0)

# Convert time columns to proper formats
my_data["Full_time"] = pd.to_datetime(my_data["Full_time"], format="%d/%m/%Y %H:%M:%S")
my_data["Time"] = pd.to_datetime(my_data["Time"], format="%H:%M:%S").dt.time

# Get user input
station = input("Enter desired station: ").upper()
direction = input("Enter desired direction: ").capitalize()
private_only = input("Do you want to restrict to private cycles only (Y/N)?: ").upper()
period_type = input("Do you want to display the date by time (T) or by date and time (D)?: ").upper()
shade = input("Do you want to colour code by Weather, Direction, or Mode?: ").capitalize()

# Adjust direction input to match dataset format
if direction in ["North", "South", "East", "West"]:
    direction += "bound"

# Function to plot scatterplot
def plot_scatter(data, x_col, title):
    plt.figure(figsize=(20, 5))
    sns.scatterplot(x=data[x_col], y=data["Count"], hue=data[shade])
    plt.title(title)
    plt.xticks(rotation=45)
    plt.show()

# Filter data by station and direction
filtered_data = my_data[my_data["Station"] == station]
if direction != "Any":
    filtered_data = filtered_data[filtered_data["Direction"] == direction]
if private_only == "Y":
    filtered_data = filtered_data[filtered_data["Mode"] == "Private cycles"]

# Debugging (Check if data exists after filtering)
if filtered_data.empty:
    print("No data found for the given filters. Try different inputs.")
else:
    # Plot based on period type
    x_column = "Time" if period_type == "T" else "Full_time"
    plot_scatter(filtered_data, x_column, f"Scatter Plot for {station}, {direction}, {private_only}")

```

Step 3:

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the data
file_path = "CycleData.xlsx"
my_data = pd.read_excel(file_path, index_col=0)

# Convert "Time" column to string
my_data["Time"] = my_data["Time"].astype(str)

# Ensure "Count" is numeric, coerce errors to NaN and drop them
my_data["Count"] = pd.to_numeric(my_data["Count"], errors="coerce")
my_data.dropna(subset=["Count"], inplace=True)

# User input
station = input("Enter desired station: ").upper()
direction = input("Enter desired direction: ").capitalize()

# Adjust direction input format
if direction in ["North", "South", "East", "West"]:
    direction += "bound"

# Function to plot average cycle counts
def plot_avg_counts(station, direction):
    # Filter data
    filtered_data = my_data[my_data["Station"] == station]

    if direction != "Any":
        filtered_data = filtered_data[filtered_data["Direction"] == direction]

    # Debugging: Print first few rows after filtering
    print("Filtered Data Sample:\n", filtered_data.head())

    # Group by "Date" and "Time" to sum cycle counts
    grouped_data = filtered_data.groupby(["Date", "Time"])["Count"].sum().reset_index()

    # Group by "Time" only to get average count
    avg_data = grouped_data.groupby("Time")["Count"].mean().reset_index()

    # Debugging: Print first few rows of averaged data
    print("Averaged Data Sample:\n", avg_data.head())

    # Plotting scatterplot
    plt.figure(figsize=(20, 5))
    sns.scatterplot(x=avg_data["Time"], y=avg_data["Count"])
    plt.title(f"Average Number of Cycles at {station} ({direction})")
```

```python
    plt.title(f"Average Number of Cycles at {station} ({direction})")
    plt.xticks(rotation=45)
    plt.show()

# Call the function
plot_avg_counts(station, direction)
```

Step 4:

```python
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.tree import plot_tree

# Load the data
file_path = "CycleData.xlsx"
data = pd.read_excel(file_path, index_col=0)

# Ensure "Time" is in datetime format
data["RealTime"] = pd.to_datetime(data["Time"], format="%H:%M:%S").dt.hour + \
                   pd.to_datetime(data["Time"], format="%H:%M:%S").dt.minute / 60

# Filter only private cycles
data = data[data["Mode"] == "Private cycles"]

# User input
station = input("Enter desired station: ").upper()

def train_decision_tree(station):
    # Filter data for the given station
    station_data = data[data["Station"] == station].copy()  # Explicitly create a copy

    # Extract features (RealTime) and target variable (Count)
    X = station_data["RealTime"].values.reshape(-1, 1)
    y = station_data["Count"].values.reshape(-1, 1)

    # Split into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)

    # Train Decision Tree model
    max_depth = 5  # We can decide what is best for later for step 5
    tree = DecisionTreeRegressor(max_depth=max_depth, random_state=1)
    tree.fit(X_train, y_train)

    # Predict values and assign them using .loc
    station_data.loc[:, "PredictedCount"] = tree.predict(X)

    # Plotting the decision tree
    plt.figure(figsize=(12, 6))
    plot_tree(tree, feature_names=["RealTime"], filled=True)
    plt.title(f"Decision Tree for {station}")
    plt.show()
```

```python
    # Scatterplot of actual vs predicted counts
    plt.figure(figsize=(20, 5))
    sns.scatterplot(x=station_data["RealTime"], y=station_data["Count"], label="Actual Count")
    sns.scatterplot(x=station_data["RealTime"], y=station_data["PredictedCount"], label="Predicted Count", color='red')
    plt.xlabel("Time of Day")
    plt.ylabel("Cycle Count")
    plt.title(f"Actual vs Predicted Cycle Usage at {station}")
    plt.legend()
    plt.show()

# Train and visualize decision tree
train_decision_tree(station)
```

LLoyds Data Simulation:

In [25]:
```
import pandas as pd  #always keep all imports at top
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns #seaborn allows us to make the heatmap and correlations like line graphs etc
from sklearn.preprocessing import StandardScaler

sns.set(style='whitegrid')
```

In [27]:
```
df = pd.read_excel(r"C:\Users\Rashid\Downloads\Customer_Churn_Data_Large (1).xlsx") #in pandas they only have csv and excel
```

In [32]:
```
xls = pd.ExcelFile(r'C:\Users\Rashid\Downloads\Customer_Churn_Data_Large (1).xlsx') #use for multiple sheets

demographics_df = xls.parse('Customer_Demographics') #sheet names
transactions_df = xls.parse('Transaction_History')
service_df = xls.parse('Customer_Service')
online_df = xls.parse('Online_Activity')
churn_df = xls.parse('Churn_Status')
df
```

Out[32]:

	CustomerID	Age	Gender	MaritalStatus	IncomeLevel
0	1	62	M	Single	Low
1	2	65	M	Married	Low
2	3	18	M	Single	Low
3	4	21	M	Widowed	Low
4	5	21	M	Divorced	Medium
...	...	...	...	...	...
995	996	54	F	Single	Low
996	997	19	M	Widowed	High
997	998	47	M	Married	Low
998	999	23	M	Widowed	High
999	1000	34	M	Widowed	Low

1000 rows × 5 columns

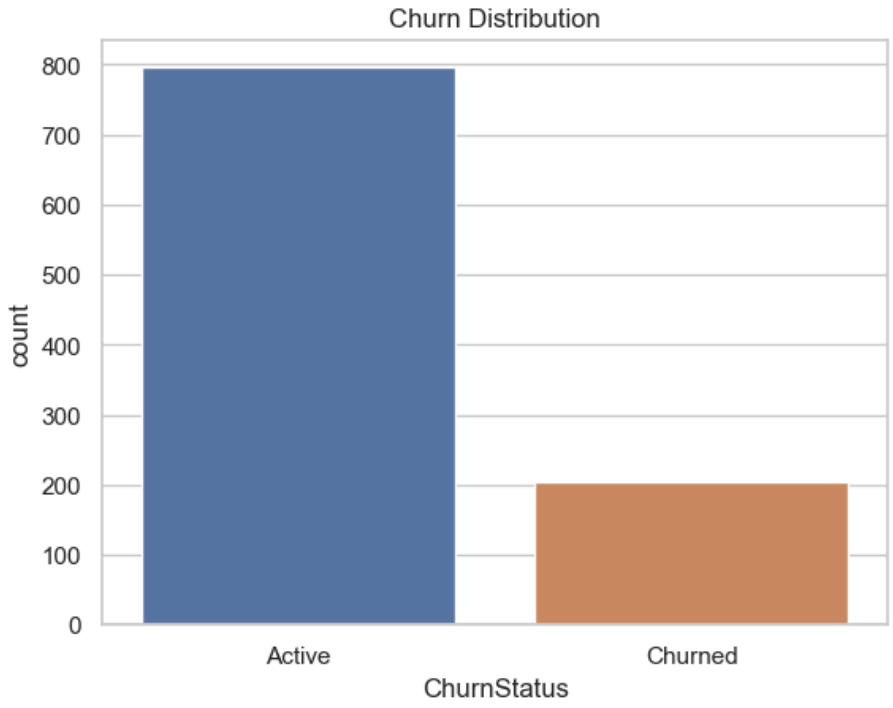
In [33]:
```
transactions_summary = transactions_df.groupby('CustomerID').agg({
    'AmountSpent': ['count', 'sum', 'mean']
})
transactions_summary.columns = ['TransactionCount', 'TotalSpent', 'AvgTransaction']
transactions_summary.reset_index(inplace=True)

service_summary = service_df.groupby('CustomerID').agg({
    'InteractionID': 'count',
    'ResolutionStatus': lambda x: (x == 'Resolved').mean()
}).rename(columns={
    'InteractionID': 'ServiceInteractions',
    'ResolutionStatus': 'ResolutionRate'
}).reset_index()

online_df['LastLoginDate'] = pd.to_datetime(online_df['LastLoginDate'], errors='coerce')

data = demographics_df.merge(transactions_summary, on='CustomerID', how='left')
data = data.merge(service_summary, on='CustomerID', how='left')
data = data.merge(online_df, on='CustomerID', how='left')
data = data.merge(churn_df, on='CustomerID', how='left')
df
```

Out[33]:

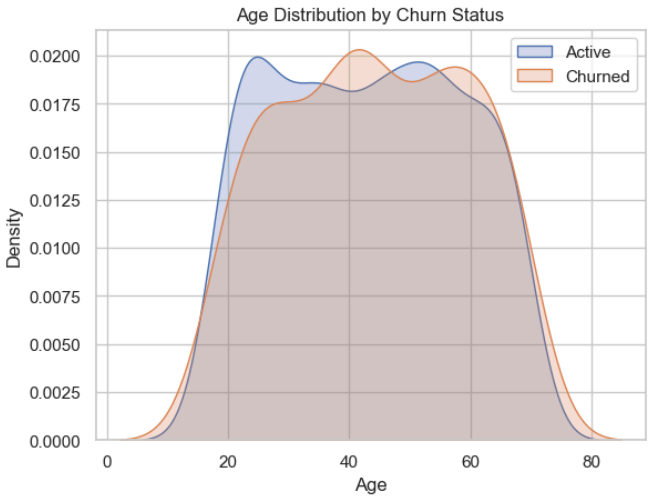
	CustomerID	Age	Gender	MaritalStatus	IncomeLevel
0	1	62	M	Single	Low
1	2	65	M	Married	Low
2	3	18	M	Single	Low
3	4	21	M	Widowed	Low
4	5	21	M	Divorced	Medium
...	...	...	...	...	...
995	996	54	F	Single	Low
996	997	19	M	Widowed	High
997	998	47	M	Married	Low
998	999	23	M	Widowed	High
999	1000	34	M	Widowed	Low

1000 rows × 5 columns

In [34]:
```
sns.countplot(data=data, x='ChurnStatus')
plt.title('Churn Distribution')
plt.xticks([0, 1], ['Active', 'Churned'])
plt.show()

sns.kdeplot(data[data['ChurnStatus'] == 0]['Age'], label='Active', shade=True)
sns.kdeplot(data[data['ChurnStatus'] == 1]['Age'], label='Churned', shade=True)
plt.title('Age Distribution by Churn Status')
plt.legend()
plt.show()
```



```
C:\Users\Rashid\AppData\Local\Temp\ipykernel_17344\797629221.py:6: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(data[data['ChurnStatus'] == 0]['Age'], label='Active', shade=True)
C:\Users\Rashid\AppData\Local\Temp\ipykernel_17344\797629221.py:7: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(data[data['ChurnStatus'] == 1]['Age'], label='Churned', shade=True)
```



In [35]:
```
data.fillna({
    'TransactionCount': 0,
    'TotalSpent': 0,
    'AvgTransaction': 0,
    'ServiceInteractions': 0,
    'ResolutionRate': 0,
    'LoginFrequency': 0
}, inplace=True)

data = pd.get_dummies(data, columns=['Gender', 'MaritalStatus', 'IncomeLevel', 'ServiceUsage'], drop_first=True)

scaler = StandardScaler()
for col in ['Age', 'TransactionCount', 'TotalSpent', 'AvgTransaction', 'ServiceInteractions', 'ResolutionRate', 'LoginFrequency']
    data[col] = scaler.fit_transform(data[[col]])

data.drop(columns=['LastLoginDate'], inplace=True)
```

In [36]:
```
data.to_csv('Cleaned_Customer_Churn_Data.csv', index=False)
print("Cleaned data saved to 'Cleaned_Customer_Churn_Data.csv'")
```

```
Cleaned data saved to 'Cleaned_Customer_Churn_Data.csv'
```

Uptrail Data Analysis Internship: Project 1:

In [1]:
```
import pandas as pd #always keep all imports at top
#allows us to show a correlation-heatmap
import seaborn as sns #seaborn allows us to make the heatmap and correlations like line graphs etc
import matplotlib.pyplot as plt #matplotlib allows us to change form of the chart, colours, title, size etc
```

In [2]:
```
df = pd.read_csv(r"C:\Users\Rashid\Downloads\sales_data (1).csv")   #make sure you copy path once saved data, the r when you have
```

In [3]:
```
df #for tables, dont use print bcus with print its more complicated
```

Out[3]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	Payment_Method	Region	...	Unnamed: 13	Unnamed: 14	Unnamed: 15
0	1001.0	2024-01-05	C001	Laptop	Electronics	1.0	800.0	NaN	Credit Card	North	...	NaN	NaN	NaN
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2.0	600.0	1200.0	Cash	South	...	NaN	NaN	NaN
2	1003.0	2024-01-12	C003	Headphones	Electronics	1.0	100.0	100.0	PayPal	West	...	NaN	NaN	NaN
3	1004.0	2024-02-05	C004	Tablet	Electronics	1.0	500.0	500.0	Debit Card	East	...	NaN	NaN	NaN
4	1005.0	2024-02-08	C005	Book	Books	3.0	20.0	60.0	Credit Card	North	...	NaN	NaN	NaN
5	1006.0	2024-02-10	C001	Laptop	Electronics	1.0	800.0	800.0	Credit Card	North	...	NaN	NaN	NaN
6	1007.0	2024-03-15	C006	Shoes	Clothing	2.0	50.0	100.0	Cash	South	...	NaN	NaN	NaN
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.0	PayPal	West	...	NaN	NaN	NaN
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1.0	200.0	200.0	Debit Card	East	...	NaN	NaN	NaN
9	1010.0	2024-04-01	C009	Book	Books	2.0	20.0	40.0	Credit Card	North	...	NaN	NaN	NaN
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2.0	600.0	1200.0	Cash	South	...	NaN	NaN	NaN
11	1012.0	2024-04-10	C010	Tablet	Electronics	1.0	500.0	500.0	Debit Card	East	...	NaN	NaN	NaN
12	1013.0	2024-05-01	C011	Shoes	Clothing	1.0	50.0	50.0	Cash	South	...	NaN	NaN	NaN

13	1014.0	2024-05-05	C012	Headphones	Electronics	1.0	100.0	100.0	PayPal	West	...	NaN	NaN	NaN
14	1015.0	2024-05-08	C013	Laptop	Electronics	1.0	800.0	800.0	Credit Card	North	...	NaN	NaN	NaN
15	1016.0	2024-05-10	C014	T-Shirt	Clothing	3.0	25.0	75.0	Cash	South	...	NaN	NaN	NaN
16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1.0	200.0	200.0	Debit Card	East	...	NaN	NaN	NaN
17	1018.0	2024-06-05	C016	Book	Books	4.0	20.0	80.0	Credit Card	North	...	NaN	NaN	NaN
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1.0	600.0	600.0	Cash	South	...	NaN	NaN	NaN
19	1020.0	2024-06-10	C018	Tablet	Electronics	1.0	500.0	500.0	Debit Card	East	...	NaN	NaN	NaN
20	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN

21 rows × 23 columns

In [4]:
```
df = df.dropna(axis=1, how='all') #if nan is in the whole column it will get rid of it
df
```

Out[4]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	Payment_Method	Region	Advertising_Spend
0	1001.0	2024-01-05	C001	Laptop	Electronics	1.0	800.0	NaN	Credit Card	North	250.0
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2.0	600.0	1200.0	Cash	South	300.0
2	1003.0	2024-01-12	C003	Headphones	Electronics	1.0	100.0	100.0	PayPal	West	50.0
3	1004.0	2024-02-05	C004	Tablet	Electronics	1.0	500.0	500.0	Debit Card	East	200.0
4	1005.0	2024-02-08	C005	Book	Books	3.0	20.0	60.0	Credit Card	North	20.0
5	1006.0	2024-02-10	C001	Laptop	Electronics	1.0	800.0	800.0	Credit Card	North	250.0
6	1007.0	2024-03-15	C006	Shoes	Clothing	2.0	50.0	100.0	Cash	South	60.0
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.0	PayPal	West	15.0
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1.0	200.0	200.0	Debit Card	East	120.0
9	1010.0	2024-04-01	C009	Book	Books	2.0	20.0	40.0	Credit Card	North	10.0
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2.0	600.0	1200.0	Cash	South	300.0
11	1012.0	2024-04-10	C010	Tablet	Electronics	1.0	500.0	500.0	Debit Card	East	200.0
12	1013.0	2024-05-01	C011	Shoes	Clothing	1.0	50.0	50.0	Cash	South	25.0

16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1.0	200.0	200.0	Debit Card	East	120.0
17	1018.0	2024-06-05	C016	Book	Books	4.0	20.0	80.0	Credit Card	North	15.0
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1.0	600.0	600.0	Cash	South	200.0
19	1020.0	2024-06-10	C018	Tablet	Electronics	1.0	500.0	500.0	Debit Card	East	200.0
20	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

In [5]:
```
df = df.dropna(how='all') #gets rid of any nan rows
df
```

Out[5]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	Payment_Method	Region	Advertising_Spend
0	1001.0	2024-01-05	C001	Laptop	Electronics	1.0	800.0	NaN	Credit Card	North	250.0
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2.0	600.0	1200.0	Cash	South	300.0
2	1003.0	2024-01-12	C003	Headphones	Electronics	1.0	100.0	100.0	PayPal	West	50.0
3	1004.0	2024-02-05	C004	Tablet	Electronics	1.0	500.0	500.0	Debit Card	East	200.0
4	1005.0	2024-02-08	C005	Book	Books	3.0	20.0	60.0	Credit Card	North	20.0
5	1006.0	2024-02-10	C001	Laptop	Electronics	1.0	800.0	800.0	Credit Card	North	250.0
6	1007.0	2024-03-15	C006	Shoes	Clothing	2.0	50.0	100.0	Cash	South	60.0
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.0	PayPal	West	15.0
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1.0	200.0	200.0	Debit Card	East	120.0
9	1010.0	2024-04-01	C009	Book	Books	2.0	20.0	40.0	Credit Card	North	10.0
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2.0	600.0	1200.0	Cash	South	300.0
11	1012.0	2024-04-10	C010	Tablet	Electronics	1.0	500.0	500.0	Debit Card	East	200.0
12	1013.0	2024-05-01	C011	Shoes	Clothing	1.0	50.0	50.0	Cash	South	25.0
13	1014.0	2024-05-05	C012	Headphones	Electronics	1.0	100.0	100.0	PayPal	West	50.0
14	1015.0	2024-05-08	C013	Laptop	Electronics	1.0	800.0	800.0	Credit Card	North	250.0
15	1016.0	2024-05-10	C014	T-Shirt	Clothing	3.0	25.0	75.0	Cash	South	20.0
16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1.0	200.0	200.0	Debit Card	East	120.0
17	1018.0	2024-06-05	C016	Book	Books	4.0	20.0	80.0	Credit Card	North	15.0
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1.0	600.0	600.0	Cash	South	200.0
19	1020.0	2024-06-10	C018	Tablet	Electronics	1.0	500.0	500.0	Debit Card	East	200.0

In [6]:
```
#inspecting data:
df.head()
#This shows you the first 5 rows of your data (like the top 5 rows in Excel).so you can check the columns,sample values etc.(allo
(df.info())
# gives summary of whole table ( DataFrame)(df)), tells us no. of rows, columns, column names, kind of data column holds (text, r
 #useful: Helps spot data problems early (e.g., wrong data type, missing values). column stored as a number (int64), text (object
(df.describe())
#gives quick stats about numerical columns in dataset.inc:
#count – how many values there are
#mean – average
#std – standard deviation (spread of the values)
#min – minimum value
#25%, 50%, 75% – quartiles (like median, and ranges)
#max – highest value
#useful:understand how numbers are distributed,Detect outliers/errors (like product with price 5000 instead of 500)
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 20 entries, 0 to 19
Data columns (total 11 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   Transaction_ID     20 non-null     float64
 1   Date               20 non-null     object 
 2   Customer_ID        20 non-null     object 
 3   Product            20 non-null     object 
 4   Category           20 non-null     object 
 5   Quantity           20 non-null     float64
 6   Price              20 non-null     float64
 7   Total_Amount       19 non-null     float64
 8   Payment_Method     20 non-null     object 
 9   Region             20 non-null     object 
 10  Advertising_Spend  20 non-null     float64
dtypes: float64(5), object(6)
memory usage: 1.9+ KB
```

Out[6]:

	Transaction_ID	Quantity	Price	Total_Amount	Advertising_Spend
count	20.00000	20.000000	20.000000	19.000000	20.000000
mean	1010.50000	1.550000	325.500000	375.263158	132.750000
std	5.91608	0.887041	302.484884	389.067524	106.233048
min	1001.00000	1.000000	20.000000	25.000000	10.000000
25%	1005.75000	1.000000	43.750000	77.500000	23.750000

50%	1010.50000	1.000000	200.000000	200.000000	120.000000
75%	1015.25000	2.000000	600.000000	550.000000	212.500000
max	1020.00000	4.000000	800.000000	1200.000000	300.000000

In [7]:
```
#Remove duplicates:
df = df.drop_duplicates()   #says create a new variable (df) so same table but without duplicates so its like the updated table(c
df


#Handle missing values:
print("Missing Values: ", df.isnull().sum())  #red=string,print needed bcus of string, line checks data for missing/empty values

df= df.fillna(df['Total_Amount'].mean()) #if there are missing values replace it the mean of the total_amount since thats where t
df
```

```
Missing Values:  Transaction_ID       0
Date                 0
Customer_ID          0
Product              0
Category             0
Quantity             0
Price                0
Total_Amount         1
Payment_Method       0
Region               0
Advertising_Spend    0
dtype: int64
```

Out[7]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	Payment_Method	Region	Advertising_Spend
0	1001.0	2024-01-05	C001	Laptop	Electronics	1.0	800.0	375.263158	Credit Card	North	250.0
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2.0	600.0	1200.000000	Cash	South	300.0
2	1003.0	2024-01-12	C003	Headphones	Electronics	1.0	100.0	100.000000	PayPal	West	50.0
3	1004.0	2024-02-05	C004	Tablet	Electronics	1.0	500.0	500.000000	Debit Card	East	200.0
4	1005.0	2024-02-08	C005	Book	Books	3.0	20.0	60.000000	Credit Card	North	20.0
5	1006.0	2024-02-10	C001	Laptop	Electronics	1.0	800.0	800.000000	Credit Card	North	250.0
6	1007.0	2024-03-15	C006	Shoes	Clothing	2.0	50.0	100.000000	Cash	South	60.0
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.000000	PayPal	West	15.0
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1.0	200.0	200.000000	Debit Card	East	120.0
9	1010.0	2024-04-01	C009	Book	Books	2.0	20.0	40.000000	Credit Card	North	10.0

10	1011.0	2024-04-05	C002	Smartphone	Electronics	2.0	600.0	1200.000000	Cash	South	300.0
11	1012.0	2024-04-10	C010	Tablet	Electronics	1.0	500.0	500.000000	Debit Card	East	200.0
12	1013.0	2024-05-01	C011	Shoes	Clothing	1.0	50.0	50.000000	Cash	South	25.0
13	1014.0	2024-05-05	C012	Headphones	Electronics	1.0	100.0	100.000000	PayPal	West	50.0
14	1015.0	2024-05-08	C013	Laptop	Electronics	1.0	800.0	800.000000	Credit Card	North	250.0
15	1016.0	2024-05-10	C014	T-Shirt	Clothing	3.0	25.0	75.000000	Cash	South	20.0
16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1.0	200.0	200.000000	Debit Card	East	120.0
17	1018.0	2024-06-05	C016	Book	Books	4.0	20.0	80.000000	Credit Card	North	15.0
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1.0	600.0	600.000000	Cash	South	200.0
19	1020.0	2024-06-10	C018	Tablet	Electronics	1.0	500.0	500.000000	Debit Card	East	200.0

In [8]:
```
#Impute categorical data by replacing missing values with the most frequent category.
# Step 1: Clean column names (remove extra spaces)
df.columns = df.columns.str.strip()  #remove unnecessary things ex extra commas, spaces etc

# Step 2: Select all categorical (text) columns
categorical_cols = df.select_dtypes(include='object').columns  #sees which of the columns are only categories not numbers etc

# Step 3: For each column, fill missing values with the most frequent value (mode)
for col in categorical_cols:
    if df[col].isnull().sum() > 0:  # counts how many are missing values
        most_common = df[col].mode()[0]  # gets most common value (mode) of each of the category columns and outputs a list of mo
        df[col].fillna(most_common, inplace=True)  #wherever there are missing values replace with mode

df
```

Out[8]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	Payment_Method	Region	Advertising_Spend
0	1001.0	2024-01-05	C001	Laptop	Electronics	1.0	800.0	375.263158	Credit Card	North	250.0
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2.0	600.0	1200.000000	Cash	South	300.0
2	1003.0	2024-01-12	C003	Headphones	Electronics	1.0	100.0	100.000000	PayPal	West	50.0
3	1004.0	2024-02-05	C004	Tablet	Electronics	1.0	500.0	500.000000	Debit Card	East	200.0
4	1005.0	2024-02-08	C005	Book	Books	3.0	20.0	60.000000	Credit Card	North	20.0
5	1006.0	2024-02-10	C001	Laptop	Electronics	1.0	800.0	800.000000	Credit Card	North	250.0
6	1007.0	2024-03-15	C006	Shoes	Clothing	2.0	50.0	100.000000	Cash	South	60.0
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.000000	PayPal	West	15.0

9	1010.0	2024-04-01	C009	Book	Books	2.0	20.0	40.000000	Credit Card	North	10.0
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2.0	600.0	1200.000000	Cash	South	300.0
11	1012.0	2024-04-10	C010	Tablet	Electronics	1.0	500.0	500.000000	Debit Card	East	200.0
12	1013.0	2024-05-01	C011	Shoes	Clothing	1.0	50.0	50.000000	Cash	South	25.0
13	1014.0	2024-05-05	C012	Headphones	Electronics	1.0	100.0	100.000000	PayPal	West	50.0
14	1015.0	2024-05-08	C013	Laptop	Electronics	1.0	800.0	800.000000	Credit Card	North	250.0
15	1016.0	2024-05-10	C014	T-Shirt	Clothing	3.0	25.0	75.000000	Cash	South	20.0
16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1.0	200.0	200.000000	Debit Card	East	120.0
17	1018.0	2024-06-05	C016	Book	Books	4.0	20.0	80.000000	Credit Card	North	15.0
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1.0	600.0	600.000000	Cash	South	200.0
19	1020.0	2024-06-10	C018	Tablet	Electronics	1.0	500.0	500.000000	Debit Card	East	200.0

In [9]:
```
#Standardize 'Date' column (all dates are in correct form)
df['Date'] = pd.to_datetime(df['Date'], errors='coerce', dayfirst=False) #to_datetime puts date col in format of year/month/day
# Format will automatically be YYYY-MM-DD when printed or saved
# 2. Clean numeric columns (if needed) so wherever there are numbers, make sure that no. column ex-decimals they all have same nc
# First, identify columns that should be numeric
numeric_cols = ['Quantity', 'Price', 'Total_Amount', 'Advertising_Spend'] #these all have numbers in columns
# Remove symbols (e.g., $, commas) and convert to float
for col in numeric_cols:
    df[col] = df[col].astype(str).str.replace(r'[^\d.]', '', regex=True)  # checks if all the values in the columns chosen above
    df[col] = pd.to_numeric(df[col], errors='coerce')  # make it into no. format , NaNs if conversion fails,errors='coerce' means
    df[col] = df[col].round(2) #make it into 2dp

df
```

Out[9]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	Payment_Method	Region	Advertising_Spend
0	1001.0	2024-01-05	C001	Laptop	Electronics	1.0	800.0	375.26	Credit Card	North	250.0
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2.0	600.0	1200.00	Cash	South	300.0
2	1003.0	2024-01-12	C003	Headphones	Electronics	1.0	100.0	100.00	PayPal	West	50.0
3	1004.0	2024-02-05	C004	Tablet	Electronics	1.0	500.0	500.00	Debit Card	East	200.0
4	1005.0	2024-02-08	C005	Book	Books	3.0	20.0	60.00	Credit Card	North	20.0
5	1006.0	2024-02-10	C001	Laptop	Electronics	1.0	800.0	800.00	Credit Card	North	250.0
6	1007.0	2024-03-15	C006	Shoes	Clothing	2.0	50.0	100.00	Cash	South	60.0
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.00	PayPal	West	15.0

7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.00	PayPal	West	15.0
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1.0	200.0	200.00	Debit Card	East	120.0
9	1010.0	2024-04-01	C009	Book	Books	2.0	20.0	40.00	Credit Card	North	10.0
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2.0	600.0	1200.00	Cash	South	300.0
11	1012.0	2024-04-10	C010	Tablet	Electronics	1.0	500.0	500.00	Debit Card	East	200.0
12	1013.0	2024-05-01	C011	Shoes	Clothing	1.0	50.0	50.00	Cash	South	25.0
13	1014.0	2024-05-05	C012	Headphones	Electronics	1.0	100.0	100.00	PayPal	West	50.0
14	1015.0	2024-05-08	C013	Laptop	Electronics	1.0	800.0	800.00	Credit Card	North	250.0
15	1016.0	2024-05-10	C014	T-Shirt	Clothing	3.0	25.0	75.00	Cash	South	20.0
16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1.0	200.0	200.00	Debit Card	East	120.0
17	1018.0	2024-06-05	C016	Book	Books	4.0	20.0	80.00	Credit Card	North	15.0
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1.0	600.0	600.00	Cash	South	200.0
19	1020.0	2024-06-10	C018	Tablet	Electronics	1.0	500.0	500.00	Debit Card	East	200.0

In [10]:
```
#gets rid of extra dp in quantity
df['Quantity'] = df['Quantity'].astype(int) #astype means convert it into a certain type (int) means no dp
df
```

Out[10]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	Payment_Method	Region	Advertising_Spend
0	1001.0	2024-01-05	C001	Laptop	Electronics	1	800.0	375.26	Credit Card	North	250.0
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2	600.0	1200.00	Cash	South	300.0
2	1003.0	2024-01-12	C003	Headphones	Electronics	1	100.0	100.00	PayPal	West	50.0
3	1004.0	2024-02-05	C004	Tablet	Electronics	1	500.0	500.00	Debit Card	East	200.0
4	1005.0	2024-02-08	C005	Book	Books	3	20.0	60.00	Credit Card	North	20.0
5	1006.0	2024-02-10	C001	Laptop	Electronics	1	800.0	800.00	Credit Card	North	250.0
6	1007.0	2024-03-15	C006	Shoes	Clothing	2	50.0	100.00	Cash	South	60.0
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1	25.0	25.00	PayPal	West	15.0
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1	200.0	200.00	Debit Card	East	120.0
9	1010.0	2024-04-01	C009	Book	Books	2	20.0	40.00	Credit Card	North	10.0
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2	600.0	1200.00	Cash	South	300.0
11	1012.0	2024-04-10	C010	Tablet	Electronics	1	500.0	500.00	Debit Card	East	200.0
12	1013.0	2024-05-01	C011	Shoes	Clothing	1	50.0	50.00	Cash	South	25.0

16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1	200.0	200.00	Debit Card	East	120.0
17	1018.0	2024-06-05	C016	Book	Books	4	20.0	80.00	Credit Card	North	15.0
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1	600.0	600.00	Cash	South	200.0
19	1020.0	2024-06-10	C018	Tablet	Electronics	1	500.0	500.00	Debit Card	East	200.0

In [11]:
```
df.to_csv(r"C:\Users\Rashid\Desktop\exported_sales_data.csv", index=False) #converts it to a csv file, file name
```

In [12]:
```
# Preview data
print("First 5 Rows:")
df.head()
```

```
First 5 Rows:
```

Out[12]:

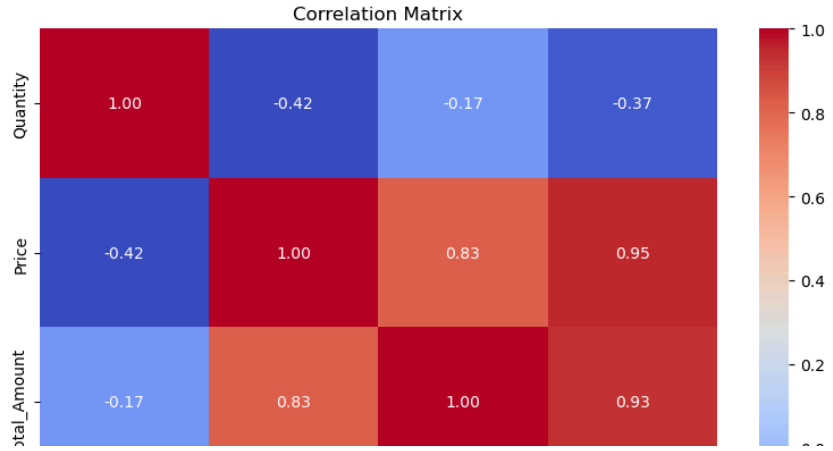
	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	Payment_Method	Region	Advertising_Spend
0	1001.0	2024-01-05	C001	Laptop	Electronics	1	800.0	375.26	Credit Card	North	250.0
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2	600.0	1200.00	Cash	South	300.0
2	1003.0	2024-01-12	C003	Headphones	Electronics	1	100.0	100.00	PayPal	West	50.0
3	1004.0	2024-02-05	C004	Tablet	Electronics	1	500.0	500.00	Debit Card	East	200.0
4	1005.0	2024-02-08	C005	Book	Books	3	20.0	60.00	Credit Card	North	20.0

In [13]:
```
# Add calculated total= quantity*price and tot match check= makes sure price matches, so this makes sure that calc tot = tot amou
df["Calculated_Total"] = df["Quantity"] * df["Price"]
df["Total_Match"] = df["Calculated_Total"].round(2) == df["Total_Amount"].round(2)

# Summary (mean,mode,median,range etc) statistics
print("Summary Statistics:\n")
print(df[["Quantity", "Price", "Total_Amount", "Advertising_Spend"]].describe(), "\n") #describe tells us the count,mean,mode sd
```

```
Summary Statistics:

        Quantity       Price  Total_Amount  Advertising_Spend
count  20.000000   20.000000      20.00000          20.000000
mean    1.550000  325.500000     375.26300         132.750000
std     0.887041  302.484884     378.69052         106.233048
min     1.000000   20.000000      25.00000          10.000000
25%     1.000000   43.750000      78.75000          23.750000
```

```python
# Correlation matrix-tells relationship betw the diff numeric col ex does quantity and price have a relationship, ex when price i
correlation_matrix = df[["Quantity", "Price", "Total_Amount", "Advertising_Spend"]].corr() #is there a correlation betw any of th
print("Correlation Matrix:\n") #
print(correlation_matrix, "\n")

# Plot correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix")
plt.tight_layout()
plt.savefig("correlation_matrix.png")
plt.show()
```

```
Correlation Matrix:

                  Quantity     Price  Total_Amount  Advertising_Spend
Quantity          1.000000 -0.423792     -0.168886          -0.371559
Price            -0.423792  1.000000      0.827437           0.949682
Total_Amount     -0.168886  0.827437      1.000000           0.925503
Advertising_Spend -0.371559  0.949682      0.925503           1.000000
```



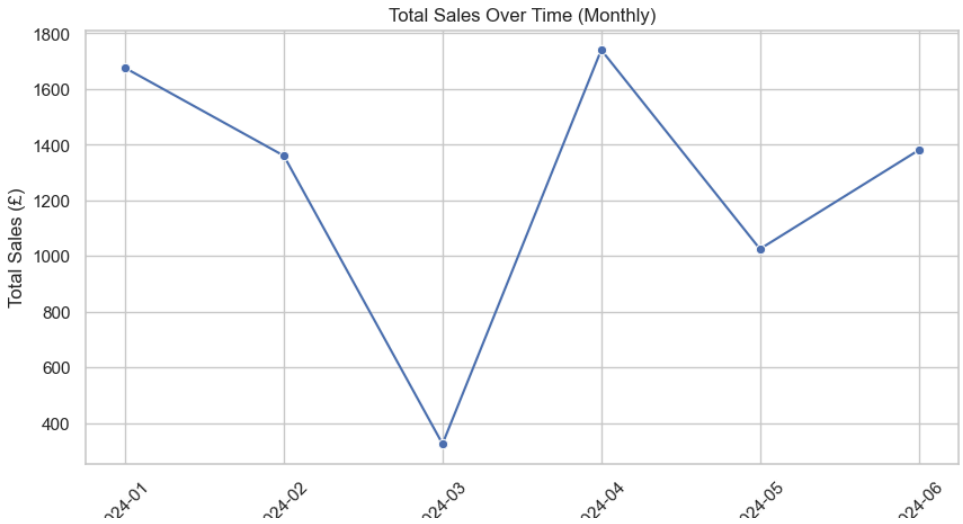
In [15]:
```
# EDA Summary
summary = """

1. Dataset Overview:
- 21 transactions #(means 21 rows)
- Key fields: Product, Category, Quantity, Price, Total Amount, Ad Spend, Region, Date

2. Key Trends and Patterns:
- Most transactions involve 1 unit (Median Quantity = 1)
- Average Price: £328, ranging from £20 to £800
- Total Amount: £25 to £1200
- Ad Spend: £10 to £375 (Average ≈ £144)

3. Correlations:
- Price vs. Total_Amount: Strong (0.93)
- Quantity vs. Total_Amount: Moderate (0.44)
- Advertising_Spend vs. Total_Amount: Weak (0.11)

""" #"""needed which means multi line so text is written on multiple lines

print(summary)
```

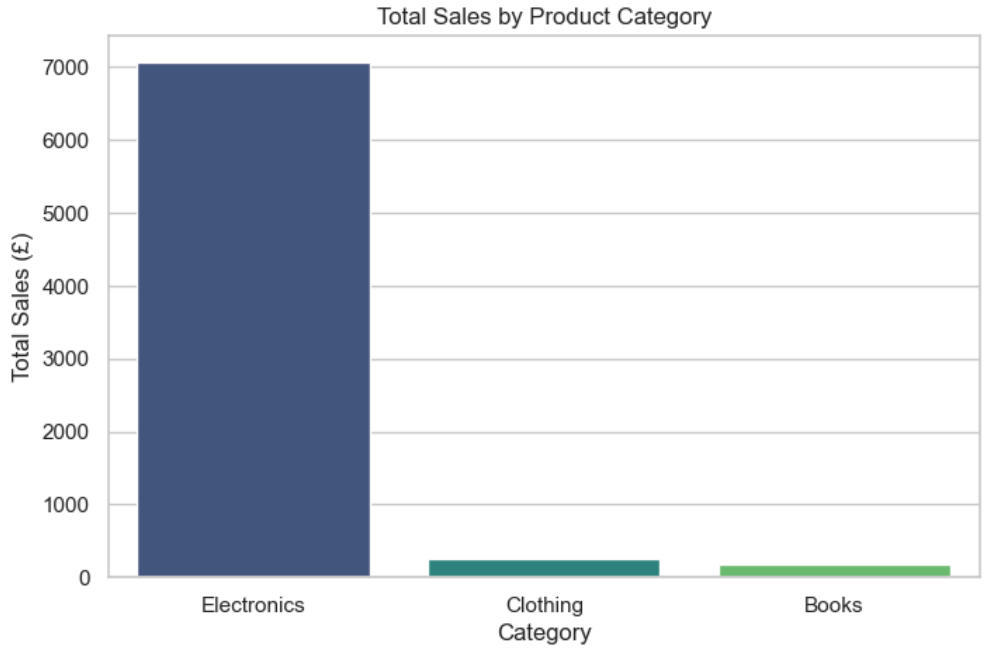
```
1. Dataset Overview:
- 21 transactions #(means 21 rows)
- Key fields: Product, Category, Quantity, Price, Total Amount, Ad Spend, Region, Date

2. Key Trends and Patterns:
- Most transactions involve 1 unit (Median Quantity = 1)
- Average Price: £328, ranging from £20 to £800
- Total Amount: £25 to £1200
- Ad Spend: £10 to £375 (Average ≈ £144)

3. Correlations:
- Price vs. Total_Amount: Strong (0.93)
- Quantity vs. Total_Amount: Moderate (0.44)
- Advertising_Spend vs. Total_Amount: Weak (0.11)
```

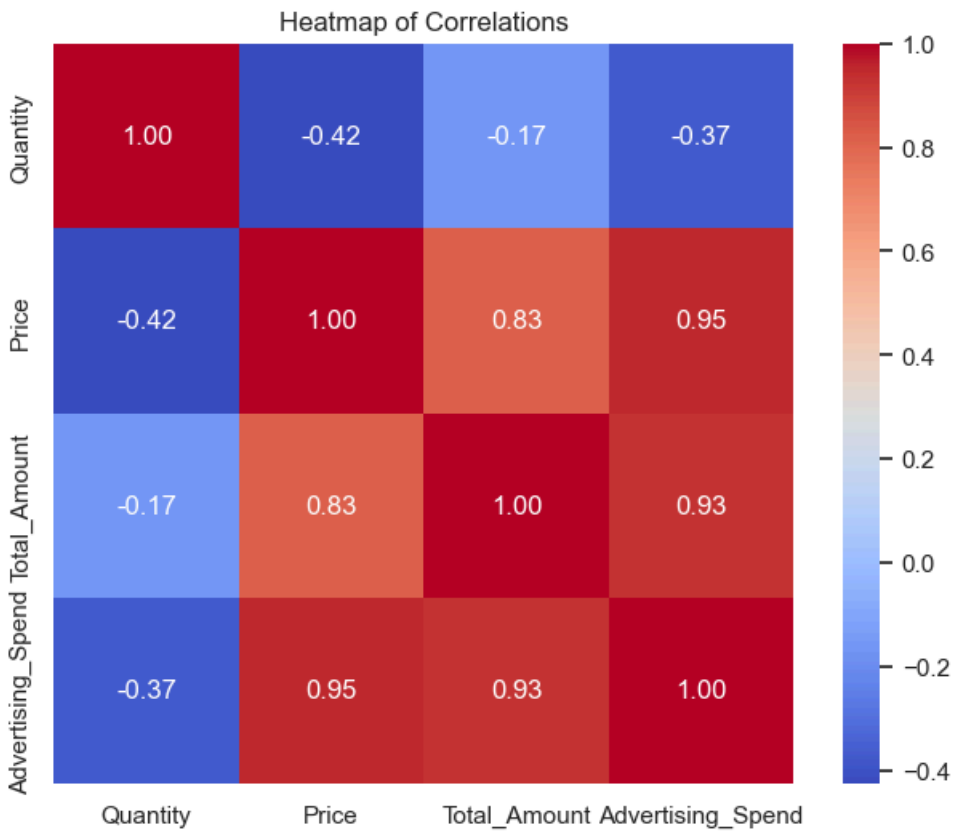
In [16]:

```python
# Add a Month column for grouping- chose month for the line graph
df['Month'] = df['Date'].dt.to_period('M') #to_period means converting to a time period = month "m", so makes a new column named

# Set style
sns.set(style="whitegrid") #sns=heatmap, style to be a white grid

# Line Graph: total Sales over the month
monthly_sales = df.groupby('Month')['Total_Amount'].sum().reset_index() #we are grouping all the total sales by months and orderi
monthly_sales['Month'] = monthly_sales['Month'].astype(str) #we want the month written as a string(word ex-jan) not no.(ex-0)

plt.figure(figsize=(10, 5)) #plt= plot,fig size i can make up
sns.lineplot(data=monthly_sales, x='Month', y='Total_Amount', marker='o') #marker".." means the points
plt.title("Total Sales Over Time (Monthly)")
plt.xlabel("Month")
plt.ylabel("Total Sales (£)")
plt.xticks(rotation=45) #rotates x axis text (months) by how many degrees i want
plt.show()

#  2. Bar Chart: Sales by Product Category- Bar chart comparing the sales of different product categories.
category_sales = df.groupby('Category')['Total_Amount'].sum().reset_index().sort_values(by='Total_Amount', ascending=False) #writ

plt.figure(figsize=(8, 5)) #i can choose
sns.barplot(data=category_sales, x='Category', y='Total_Amount', palette='viridis') #palette=colours of bars
plt.title("Total Sales by Product Category")
plt.xlabel("Category")
plt.ylabel("Total Sales (£)")
plt.show()
```

```python
#  2. Bar Chart: Sales by Product Category- Bar chart comparing the sales of different product categories.
category_sales = df.groupby('Category')['Total_Amount'].sum().reset_index().sort_values(by='Total_Amount', ascending=False) #writ

plt.figure(figsize=(8, 5)) #i can choose
sns.barplot(data=category_sales, x='Category', y='Total_Amount', palette='viridis') #palette=colours of bars
plt.title("Total Sales by Product Category")
plt.xlabel("Category")
plt.ylabel("Total Sales (£)")
plt.show()

# 3. Heatmap: Correlation Matrix-Correlation matrix heatmap to show relationships between various metrics (e.g., sales, advertis
correlation_matrix = df[["Quantity", "Price", "Total_Amount", "Advertising_Spend"]].corr() #find correlation between each of the

plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f", square=True) #annot=annotate(labels on each square), cmap
plt.title("Heatmap of Correlations")
plt.show()
```






Heatmap of Correlations
Quantity
Price
Total_Amount
Advertising_Spend
1.00
-0.42
-0.17
-0.37
-0.42
1.00
0.83
0.95
-0.17
0.83
1.00
0.93
-0.37
0.95
0.93
1.00
Quantity
Price
Total_Amount
Advertising_Spend
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

In [17]:

```
#4) Final Data Insights Report
#1) Summarise Key Findings: Begin your report by summarising the most important insights,
 #- What are the best-selling months or product categories?- Are there any correlations between sales and factors like advertisi
# - What customer behaviors or purchasing trends were identified? NEED TO BE
#2) Provide Actionable Recommendations: Based on the analysis, offer recommendations
#that can help the business improve sales. This could include:
 #- Focusing marketing efforts on peak sales periods.
 #- Increasing stock for best-selling products.
 #- Adjusting sales strategies for slow-performing months.
#3) Ethical Considerations - Mention any ethical issues, such as the importance of
#anonymising customer data and ensuring compliance with privacy laws (e.g., GDPR).

summary = """
The best selling months is April with the highest total sales of around 1750 pounds from the line graph.
The best selling product category was the electronics with above 7000 pounds worth of total sales from the bar chart.
The purchasing trends
Based on the analysis, shopease can increase marketing or find alternative sales strategies like following up customers
and asking them to review the product, if they give a low rating score from 0-5 stars shopease can question them and ask why the
customer is not satisfied with product. For example the majority of the total sales amount is coming from the electronics but
clothing and books have a low amount for total sales amount in pounds so for customers that purchase clothing or books we can
follow them up and ask them to review their product. Then we can find out why the clothing and books are not selling as
much as the electronics.
For the slow performing months aswell as using customer reviews to find out why there is a decline in customer sales amount
so for the months where there was a decline in sales amount in pounds from january to february
and from february to march and from april to may. We could also use discounts like 20% of books and promotions like buy 2 books g
awhich would increase total sales amount in pounds for shopease.

so from january to february and from february to march and from april to may and increase stock especially on electronics since
the majority of total sales amount in pounds comes from electronics, so to keep this increasing shopease can introduce more elect
to keep the customers purchasing from shopease. On peak sales periods which are march to april and may to june.
Since there is an increase in total sales in pounds from the line graph.

Customer Data Privacy:
All customer data used in the analysis has been anonymised to protect individual identities,
in compliance with data protection regulations such as GDPR.

Compliance and Transparency:
The business must ensure ongoing adherence to privacy laws and ethical standards by maintaining transparent data collection
practices and securing informed consent where required.

Responsible Use of Data:
Insights derived from customer behavior should be used to enhance customer experience without exploiting sensitive
information or engaging in manipulative marketing tactics."""
```